



FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065 (INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

**IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED
REASONING (RBR), AND CASE-BASED REASONING (CBR)**

Student Name	Iana Kim Afilleje
Section:	AN42
Professor:	Ms. Crisola Tan
Date:	9/30/2026

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge.

Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.
4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

Materials Needed

1. Hardware

- A laptop or desktop computer
- Stable internet connection (for research, documentation, submission)

2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4_lastname1.pdf**
(Example: TA2_Patilano.pdf)
11. Submit the **python file** and **PDF** file.

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

Example structure to follow:

```
facts = {  
    "temperature": 32,  
    "is_raining": False,  
    "humidity": 85,  
}
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF–THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):  
    if facts["temperature"] > 30:  
        return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. Reads the knowledge base
2. Checks each rule
3. Returns all actions inferred

Task 2C — Display Output

Example [Program should print:]

Rule-Based Reasoning Actions:

- Turn on the aircon
- Bring umbrella

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, each containing:

- A set of problem features
- A matching solution

Example:

```
case_base = [  
    {"problem": {"won't turn on": True, "noise": False},  
     "solution": "Check power cable."}  
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

Task 3E — Retain Step

Add the new case to the case base.

PART 4 — Comparative Reflection

Write a short paragraph answering:

1. Which system (RBR or CBR) was easier to implement? Why?
2. Which is more flexible in solving unstructured problems?
3. Which is better for situations with continuously changing data?
4. Examples of real-world applications for both.

Your Code - Laboratory Tasks

PART 1: Knowledge Representation (KR) Setup

```
facts = {  
    "temperature": 34,          # Category 1: Temperature (Numeric °C)  
    "is_raining": True,        # Category 2: Environment (Boolean)  
    "humidity": 88,            # Category 1: Temperature/Moisture (Numeric %)  
    "time_of_day": "Afternoon", # Category 3: Status/Time (String)  
    "wind_speed": 18           # Category 2: Environment (Numeric km/h)  
}
```

```

# PART 2: Rule-Based Reasoning (RBR)
def rule_hot_weather(f):
    if f["temperature"] > 30:
        return "Action: Turn on the air conditioner."

def rule_rainy_weather(f):
    if f["is_raining"]:
        return "Action: Bring an umbrella when going outside."

def rule_high_humidity(f):
    if f["humidity"] > 80 and f["temperature"] > 28:
        return "Action: Set air conditioner mode to Dehumidify."

def rule_afternoon_heat(f):
    if f["time_of_day"] == "Afternoon" and f["temperature"] > 32:
        return "Action: Close window blinds to block direct sunlight."

def inference_engine(knowledge_base):
    rules = [
        rule_hot_weather,
        rule_rainy_weather,
        rule_high_humidity,
        rule_afternoon_heat
    ]

    inferred_actions = []
    for rule in rules:
        result = rule(knowledge_base)
        if result:
            inferred_actions.append(result)

    return inferred_actions

# PART 3: Case-Based Reasoning (CBR)
# Case Base containing past problems and verified solutions
case_base = [
    {
        "case_id": "Case-1",
        "problem": {"temperature_high": True, "raining": False,
"high_humidity": True},
        "solution": "Turn on AC in dry mode and close window shades."
    },
    {
        "case_id": "Case-2",
        "problem": {"temperature_high": True, "raining": True, "high_humidity":
True},
        "solution": "Turn on AC, close windows, and stay indoors with umbrella
ready."
    },
    {
        "case_id": "Case-3",

```

```

        "problem": {"temperature_high": False, "raining": True,
"high_humidity": True},
        "solution": "Wear a light rain jacket and open ventilation vents."
    }
]

def calculate_similarity(new_problem, past_case_problem):
    matches = 0
    total_features = len(new_problem)

    for key, value in new_problem.items():
        if past_case_problem.get(key) == value:
            matches += 1

    return matches / total_features

def cbr_cycle(new_problem):
    print("\n--- CBR Step 1: RETRIEVE ---")
    best_case = None
    highest_similarity = -1.0

    for case in case_base:
        sim = calculate_similarity(new_problem, case["problem"])
        print(f"Checking {case['case_id']}: Similarity = {sim * 100:.1f}%")
        if sim > highest_similarity:
            highest_similarity = sim
            best_case = case

    print(f"\n--- CBR Step 2: REUSE ---")
    retrieved_solution = best_case["solution"]
    print(f"Retrieved Solution from [{best_case['case_id']}]: {retrieved_solution}")

    print("\n--- CBR Step 3: REVISE ---")
    # Custom adaptation logic for revision
    if new_problem.get("raining") and "umbrella" not in retrieved_solution:
        final_solution = retrieved_solution + " (Revised: Add recommendation to secure wet footwear outside.)"
    else:
        final_solution = retrieved_solution
    print(f"Final Adapted Solution: {final_solution}")

    print("\n--- CBR Step 4: RETAIN ---")
    new_case_id = f"Case-{len(case_base) + 1}"
    new_case = {
        "case_id": new_case_id,
        "problem": new_problem,
        "solution": final_solution
    }
    case_base.append(new_case)
    print(f"Successfully saved [{new_case_id}] into Case Base!")

```

```

        return final_solution

# Main Execution

def main():

    print("=====")
    print(" TA4: KNOWLEDGE REPRESENTATION, RBR, AND CBR COMPARISON ")
    print("=====")

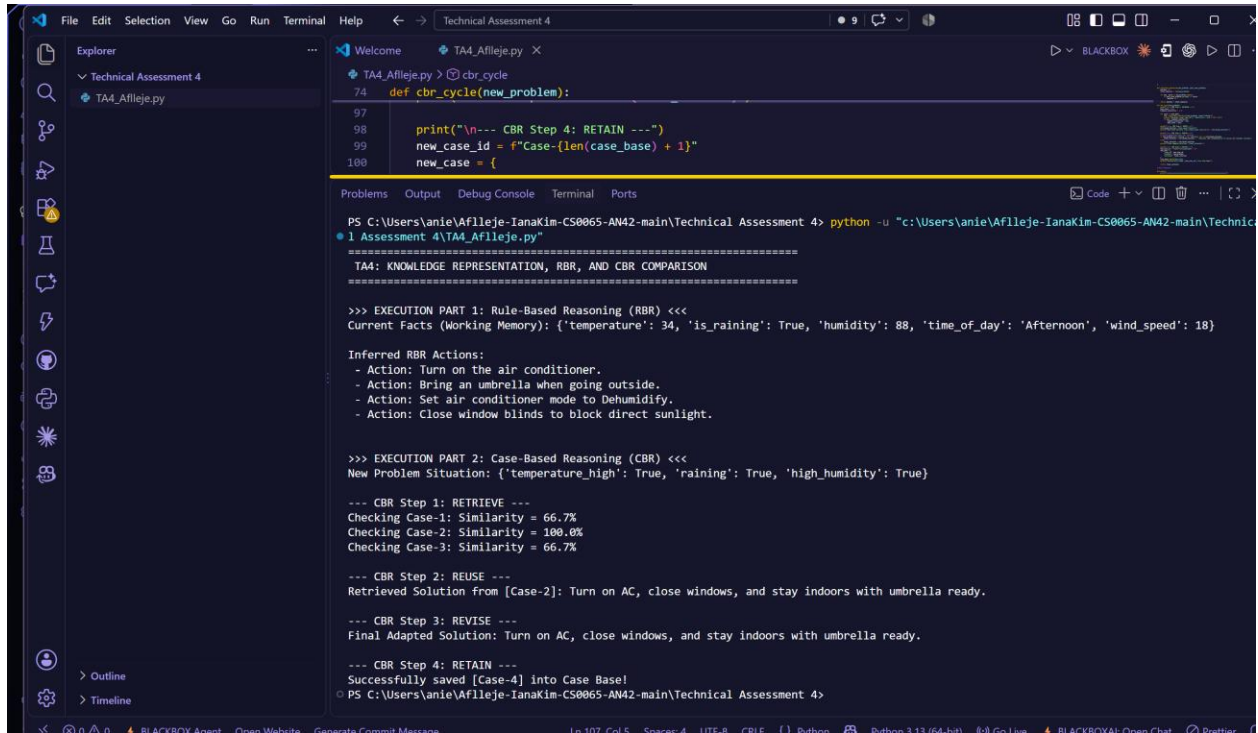
    # Run RBR
    print("\n>>> EXECUTION PART 1: Rule-Based Reasoning (RBR) <<<")
    print("Current Facts (Working Memory):", facts)
    actions = inference_engine(facts)
    print("\nInferred RBR Actions:")
    for act in actions:
        print(f" - {act}")

    # Run CBR
    print("\n\n>>> EXECUTION PART 2: Case-Based Reasoning (CBR) <<<")
    # Represent current situation as a new problem
    new_situation = {
        "temperature_high": facts["temperature"] > 30,
        "raining": facts["is_raining"],
        "high_humidity": facts["humidity"] > 80
    }
    print("New Problem Situation:", new_situation)
    cbr_cycle(new_situation)

if __name__ == "__main__":
    main()

```

Screenshots:



The screenshot displays the Visual Studio Code interface. The Explorer pane on the left shows a project named 'Technical Assessment 4' containing a file 'TA4_Afilleje.py'. The main editor window shows the code for the 'cbr_cycle' function in 'TA4_Afilleje.py'. The function is defined as follows:

```
74 def cbr_cycle(new_problem):  
97  
98     print("\n--- CBR Step 4: RETAIN ---")  
99     new_case_id = f"Case-{len(case_base) + 1}"  
100     new_case = {
```

The Output pane at the bottom shows the execution of the script. It starts with the command: `PS C:\Users\anie\Afilleje-IanaKim-CS0065-AN42-main\Technical Assessment 4> python -u "c:\Users\anie\Afilleje-IanaKim-CS0065-AN42-main\Technical Assessment 4\TA4_Afilleje.py"`. The output includes a title 'TAA: KNOWLEDGE REPRESENTATION, RBR, AND CBR COMPARISON' and two main execution parts. Part 1, 'EXECUTION PART 1: Rule-Based Reasoning (RBR)', shows current facts and inferred actions. Part 2, 'EXECUTION PART 2: Case-Based Reasoning (CBR)', shows the retrieval, reuse, and revision of cases, leading to a final adapted solution. The process concludes with 'CBR Step 4: RETAIN' and a message 'Successfully saved [Case-4] into Case Base!'.